Algorithms Homework1

Created	@November 3, 2025 9:02 PM

Завдання 1 - База даних

Мета

створення внутрішнього представлення бази студентів у пам’яті на основі CSV-файлу `students.csv`. Кожен запис містить 9 полів: ім’я, прізвище, email, дату народження, групу, рейтинг, номер телефону.

Функція `load_students()` (файл homework_shevchuk.cpp) виконує повне завантаження CSV-файлу:

1. Пропускає заголовок.
2. Читає файл порядково.
3. Для кожного рядка:
   - розділяє поля за `,`
   - видаляє зайві пропуски (`trim()`)
   - конвертує числові поля через `stoi`, `stof`
4. Створює об’єкт `Student` і додає його у `std::vector<Student>`.

У реалізації застосовано попереднє резервування (`reserve(100000)`), що зменшує кількість реалоцацій при великих обсягах даних.

Час: Парсинг кожного рядка займає O(L), де L — довжина рядка. Для N студентів: **O(N)**.

Пам’ять: Зберігається рівно N структур `Student`.

Завдання 2 - V2. Структури даних для операцій за варіантом. A:B:C = 10:100:1

Для варіанта V2 мені потрібно було зробити:

1. **findByBirthday(month, day)** — пошук студентів за днем народження
2. **changeGroupByEmail(email, newGroup)** — змінити групу за email
3. **groupWithMostSameBirthday()** — знайти групу з найбільшою кількістю студентів, які мають однаковий день народження

Співвідношення операцій: **A:B:C = 10 : 100 : 1**, тобто:

- 9.1% — пошук за днем народження
- 90.9% — зміна групи (найчастіша операція)
- 0.9% — запит на глобальний максимум

Це накладає сильні вимоги: найважливіше — швидкий доступ за email і швидке оновлення статистики груп.

різні варіанти:

- **Variant A** — unordered_map + unordered_map
- **Variant B** — map + map
- **Variant C** — на векторах

Variant A (Hash map)

Реалізація у класі `StudentDBVariantA`

Внутрішні структури

- `unordered_map<string, Student*> byEmail`

  Швидкий доступ до студента за email — O(1).

- `unordered_map<int, vector<const Student*>> byBirthday`

  Ключ: `month * 100 + day`.

Пошук за днем народження — теж O(1).

- `unordered_map<string, GroupStats> groupStats`

  Для кожної групи зберігається:

  - масив `counts[372]` — кількість студентів у групі на кожну дату;
  - `currentMax` — найбільша кількість студентів, що мають однаковий день в цій групі.

- `globalBestGroup`, `globalBestValue`

  Глобальний максимум по всіх групах, щоб не перераховувати його при кожному запиті.

Складність операцій

Операція	Складність	Пояснення
findByBirthday	**O(1)**	прямий доступ у хеш-таблиці
changeGroupByEmail	**O(1)** + O(372)	зменшити/збільшити лічильники, оновити локальний та глобальний максимум
groupWithMostSameBirthday	**O(1)**	значення зберігається

Переваги і недоліки

Плюси:

- найшвидший доступ за email;
- найшвидший пошук за днем народження.

Мінуси:

- дуже велике використання пам’яті (через кілька unordered_map).

Використання пам’яті при N=100000

Variant A ≈ **19.7 MiB** — найбільше серед усіх.

Variant B (Деревна структура (std::map))

Реалізація у `StudentDBVariantB`

Внутрішні структури

- `map<string, Student*> byEmail` — логарифмічний доступ
- `map<int, vector<const Student*>> byBirthday`
- `map<string, GroupStats> groupStats`

Складність

Операція	Складність
findByBirthday	**O(log N)**
changeGroupByEmail	**O(log N) + O(372)**
groupWithMostSameBirthday	**O(1)**

Переваги і недоліки

Плюси:

- менше пам'яті, ніж Variant A;
- впорядкованість ключів.

Мінуси:

- найповільніший із трьох (логарифмічний доступ у двох map).

Variant C (структура на векторах (найефективніший))

Реалізація `StudentDBVariant`

Головна ідея

- Замість хеш-таблиці з `int → vector` я одразу створює:

  ```
  vector<vector<const Student*>> byBirthdayBuckets;
  ```

  де індекс рахується як

  ```
  idx = (month - 1) * 31 + (day - 1)
  ```

  Це дає **ідеальний O(1)** доступ без хешування й без дерева.

Групова статистика така ж, як у A.

Складність

Операція	Складність
findByBirthday	**O(1)**
changeGroupByEmail	**O(1) + O(372)**
groupWithMostSameBirthday	**O(1)**

Переваги і недоліки

Плюси:

- найшвидший варіант;
- найменше пам'яті;
- відсутність хешування.

Мінуси:

- bucket-підхід працює лише тому, що простір дат обмежений (372 комірки).

Використання пам'яті при N=100000

Variant C ≈ **11.7 MiB** — НАЙМЕНШЕ з усіх.

Порівняння ефективності Variant A/B/C

Пропускна здатність (ops/s)

- При малих N Variant A трохи випереджає C (бо unordered_map дає максимально швидкий доступ).
- Починаючи з N ≥ 1000, Variant C стає найкращим.
- Variant B значно повільніший через `map` (O(log N)).

При N=100000

Variant	ops/s
C	**≈ 23 000**
A	≈ 20 000
B	≈ 14 000

Variant C — МАКСИМАЛЬНА продуктивність у великій базі.

Використання пам'яті

Variant	RAM usage
A	**19.7 MiB**
B	13.7 MiB
C	**11.7 MiB (найменше)**

Variant C знову найкращий.

Висновок

Критерій	Найкращий варіант
Швидкодія	**Variant C**
Пам'ять	**Variant C**
Простота реалізації	Variant A
Детермінованість	Variant B

Variant C - найкращий

Це обумовлено:

- O(1) доступом без хешів і дерев,
- компактним представленням дат,
- мінімальними накладними витратами,
- ефективним оновленням статистики груп.

Як працюють функції для завдання 2

Допоміжні функції для `benchmark.cpp`

`size_t current_rss_bytes()`

Функція вимірює **поточний обсяг зайнятої оперативної пам'яті процесом** (RSS — resident set size) у байтах.

1. Викликається `GetProcessMemoryInfo` з Windows API (бо у мене вінда).
2. Якщо виклик успішний — повертається поле `WorkingSetSize` (кількість байтів RAM, які займає процес).
3. Якщо щось пішло не так — повертається 0.

Використовується, щоб порахувати **дельту пам'яті** до й після створення структури даних.

`const std::string& rand_email(const std::vector<Student>& db, std::mt19937& gen)`

Повертає **випадковий email** зі всієї бази:

1. Створює рівномірний розподіл `dist(0, db.size()-1)`.
2. Генерує випадковий індекс.
3. Повертає `m_email` студента на цій позиції.

`void rand_birthday(const std::vector<Student>& db, int& m, int& d, std::mt19937& gen)`

Записує у змінні `m`, `d` **випадкову дату народження** з бази:

1. Аналогічно обирає випадкового студента.
2. Присвоює `m = s.m_birth_month`, `d = s.m_birth_day`.

`std::string rand_group(const std::vector<Student>& db, std::mt19937& gen)`

Повертає **випадкову назву групи** з бази: випадковий студент → його `m_group`.

`template <class Storage> size_t run_mix(...)`

Це “двигун” бенчмарку Task II — він **імітує випадкові запити** до структури даних протягом заданої кількості секунд.

Параметри:

- `Storage& storage` — конкретна реалізація (`StudentDBVariantA/B/C`);
- `db_view` — вектор студентів;
- `seconds` — тривалість тесту;
- `gen` — генератор випадкових чисел.

Алгоритм:

1. Запам’ятати час старту `start`.
2. Створити розподіл `pick(0,110)` — для випадкового вибору операції.
3. В циклі:
   - дивимось, скільки часу минуло; якщо ≥ `seconds` — виходимо;
   - генеруємо `r = pick(gen)`:
     - якщо `r < 10` → викликаємо **findByBirthday**:
       - через `rand_birthday` обираємо випадковий `(month, day)`;
       - `storage.findByBirthday(m, d)`;
     - якщо `10 ≤ r < 110` → викликаємо **changeGroupByEmail**:
       - беремо випадковий email (`rand_email`);
       - беремо випадкову групу (`rand_group`);

        - `storage.changeGroupByEmail(email, g)`;
    - інакше (r ≥ 110) → **groupWithMostSameBirthday**.
  - збільшуємо лічильник `ops`.
4. Повертаємо загальну кількість операцій `ops`.

Таким чином зберігається співвідношення **10 : 100 : 1** для трьох типів операцій.

Допоміжні функції з `homework.h / homework_shevchuk.cpp`

`inline int birthday_index(int month, int day)`

Перетворює дату на **цілий індекс** від 0 до 371:

```
return (month - 1) * 31 + (day - 1);
```

Тобто:

- усі місяці вважаються довжиною 31 день (це просто “решітка” 12×31),
- у результаті маємо 372 можливих осередки для “бакетів” днів.

Цей індекс використовується у Variant A/B/C для масивів і векторів частот.

Variant A — `StudentDBVariantA`

Конструктор `StudentDBVariantA(std::vector<Student>& all)`

Мета: **побудувати всі індекси** (по email, по дню народження та груповій статистиці).

1. Зберігає посилання на вектор `allStudents = all`.
2. Ініціалізує `globalBestValue = 0`, `globalBestGroup = ""`.
3. Резервує пам’ять для `byEmail` та `byBirthday`, щоб уникнути частих алокацій.
4. Для кожного студента `st`:

- Додає в `byEmail[st.m_email] = &st`.
- Обчислює ключ дня народження `key = month * 100 + day`.
- Додає `&st` у `byBirthday[key]`.
- Обчислює `idx = birthday_index(month, day)`.
- Береться `GroupStats& gs = groupStats[st.m_group]`.
  - Збільшує `gs.counts[idx]`.
  - Оновлює `gs.currentMax`, якщо треба.
- Якщо `gs.currentMax` > `globalBestValue`, оновлює:
  - `globalBestValue = gs.currentMax`;
  - `globalBestGroup = st.m_group`.

У результаті:

- всі індекси готові,
- ми вже знаємо групу з найбільшим збігом днів народжень.

`std::vector<const Student*> findByBirthday(int month, int day) const`

Пошук студентів за днем народження.

1. Обчислює ключ `key = month * 100 + day`.
2. Шукає його в `byBirthday`.
   - Якщо немає → повертає порожній вектор.
   - Якщо є → повертає `it→second` — вектор вказівників на студентів.

Складність: **O(1) амортизовано**.

`void changeGroupByEmail(const std::string &email, const std::string &newGroup)`

Найважливіша операція.

Змінює групу в студента за email та **оновлює групову статистику**.

1. Шукає `email` в `byEmail`.

- Якщо не знайдено → `return`.

2. Отримує `Student* st = pos->second`.
3. Зберігає `oldGroup = st->m_group`.
   - Якщо `oldGroup == newGroup` → нічого не робимо.
4. Обчислює `idx = birthday_index(st->m_birth_month, st->m_birth_day)`.

Оновлення старої групи:

1. Беремо `GroupStats& gsOld = groupStats[oldGroup]`.
2. Зменшуємо `gsOld.counts[idx]`.
3. Повторно знаходимо `gsOld.currentMax`:
   - проходимо по всіх 372 елементах і шукаємо максимум.
   - це гарантує коректність навіть після зменшення.

Оновлення нової групи:

1. Беремо `GroupStats& gsNew = groupStats[newGroup]`.
2. Збільшуємо `gsNew.counts[idx]`.
3. Якщо `gsNew.counts[idx] > gsNew.currentMax` — оновлюємо `gsNew.currentMax`.

Фактична зміна групи:

1. `st->m_group = newGroup`.

Перерахунок глобального максимуму:

1. Обнуляємо `globalBestValue` і `globalBestGroup`.
2. Проходимо по всіх `groupStats`:
   - якщо `gs.currentMax > globalBestValue`, оновлюємо:
     - `globalBestValue`;
     - `globalBestGroup`.

Складність:

- доступ до мап — O(1) амортизовано;
- перерахунок max в одній групі — O(372) ≈ константа;

- перерахунок глобального максимуму — O(G), де G — кількість груп (відносно невелике число, зазвичай ≪ N).

`std::string groupWithMostSameBirthday() const`

Просто повертає `globalBestGroup`.

- Ніяких обчислень;
- Складність **O(1)**.

Variant B — `StudentDBVariantB`

Variant B працює аналогічно A, але замість `unordered_map` всюди використовується `std::map`.

Конструктор `StudentDBVariantB(std::vector<Student>& all)`

Повністю повторює логіку Variant A:

1. Будує `byEmail`, `byBirthday`, `groupStats`.
2. Для кожного студента оновлює лічильники і `globalBestGroup`.

Різниця — всі структури на `map`, тобто:

- вставка і пошук мають складність **O(log N)**.

`findByBirthday(int month, int day) const`

1. `key = month * 100 + day`.
2. `auto it = byBirthday.find(key)`.
3. Повертає відповідний вектор або порожній.

Складність: **O(log N)** (через map).

`changeGroupByEmail(const std::string& email, const std::string& newGroup)`

Логіка 1-в-1 така ж, як у Variant A:

1. шукаємо `email` у `byEmail` (O(log N));

2. змінюємо групу;
3. коригуємо статистику `groupStats` (масив з 372 елементів);
4. перебираємо всі групи, щоб знайти глобальний максимум.

Відмінність лише в тому, що всі асоціативні доступи мають логарифмічну складність.

`groupWithMostSameBirthday() const`

Так само повертає `globalBestGroup` — O(1).

Variant C — `StudentDBVariantC`

Конструктор `StudentDBVariantC(std::vector<Student>& all)`

1. Зберігає посилання `allStudents = all`.
2. Ініціалізує `globalBestValue = 0`, `globalBestGroup = ""`.
3. Розмір масиву бакетів:

```
byBirthdayBuckets.resize(372);
```

4. Для кожного студента `st`:
   - додає в `byEmail[st.m_email] = &st` (unordered_map);
   - рахує `idx = birthday_index(month, day)` і додає `&st` у `byBirthdayBuckets[idx]`;
   - дістає `GroupStats& gs = groupStats[st.m_group]`:
     - якщо вперше бачимо цю групу, ініціалізуємо `gs.counts.assign(372, 0)`;
     - збільшуємо `gs.counts[idx]`;
     - оновлюємо `gs.currentMax`, якщо потрібно;
   - за потреби оновлюємо `globalBestGroup` і `globalBestValue`.

У підсумку:

- дні народження розкладені по фіксованих 372 бакетах,

- статистика груп порахована,
- глобальний максимум готовий.

`std::vector<const Student*> findByBirthday(int month, int day) const`

1. Обчислює `idx = birthday_index(month, day)`.
2. Перевіряє, чи idx у межах `[0, byBirthdayBuckets.size())`.
3. Якщо в межах — повертає `byBirthdayBuckets[idx]`, інакше — порожній вектор.

Складність: **O(1)**.

`void changeGroupByEmail(const std::string& email, const std::string& newGroup)`

Логіка та сама, що у A/B, але з bucket-структурою:

1. Шукає `email` в `byEmail` (unordered_map, O(1)).
2. Якщо студент знайдений:
   - бере `oldGroup`, перевіряє, чи відрізняється від `newGroup`;
   - рахує `idx = birthday_index(st->m_birth_month, st->m_birth_day)`.

Оновлення старої групи:

1. `GroupStats& gsOld = groupStats[oldGroup]`:
   - `gsOld.counts[idx]--`;
   - переобчислює `gsOld.currentMax` циклом по 372 елементах.

Оновлення нової групи:

1. `GroupStats& gsNew = groupStats[newGroup]`:
   - якщо `gsNew.counts` ще порожній — `assign(372, 0)`;
   - збільшує `gsNew.counts[idx]`;
   - оновлює `gsNew.currentMax`, якщо треба.

Оновлення поля студента та глобального максимуму:

1. `st->m_group = newGroup`;

2. Повністю перераховує `globalBestGroup` і `globalBestValue`, проходячи по всіх елементах `groupStats`.

Знову ж таки, основні витрати — це:

- O(1) доступи до unordered_map;
- O(372) на перерахунок максимуму в групі;
- O(G) на перерахунок глобального максимуму (де G — кількість груп).

`std::string groupWithMostSameBirthday() const`

Повертає `globalBestGroup` — **O(1)**.

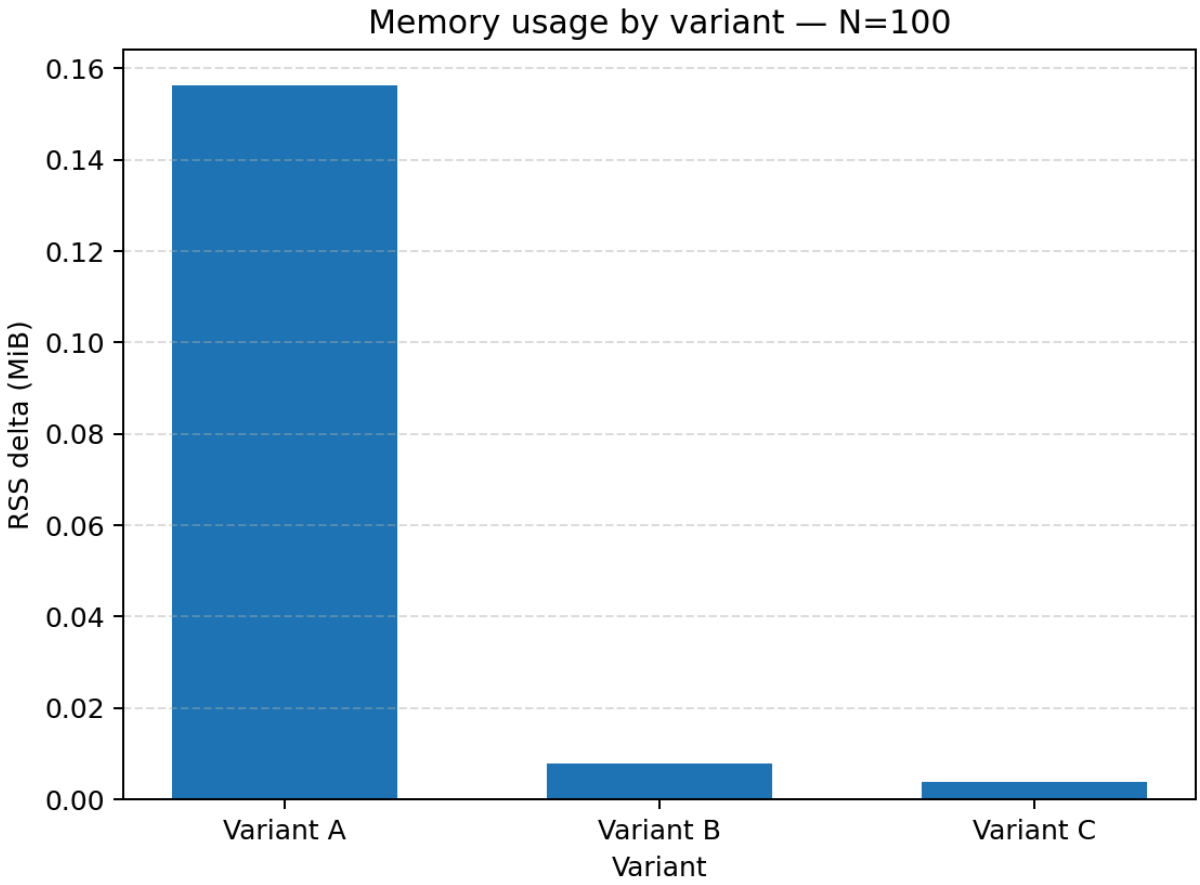
Завдання 3 - S7

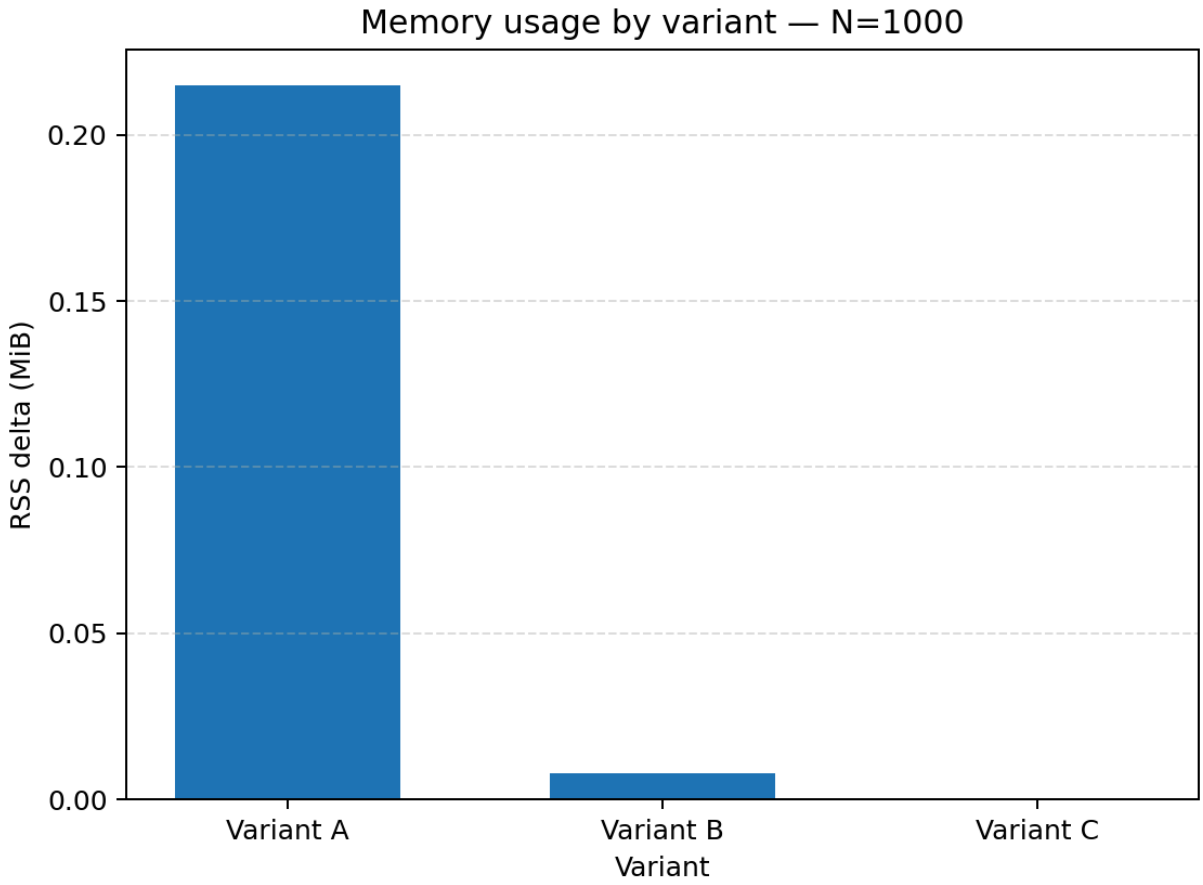
Мета

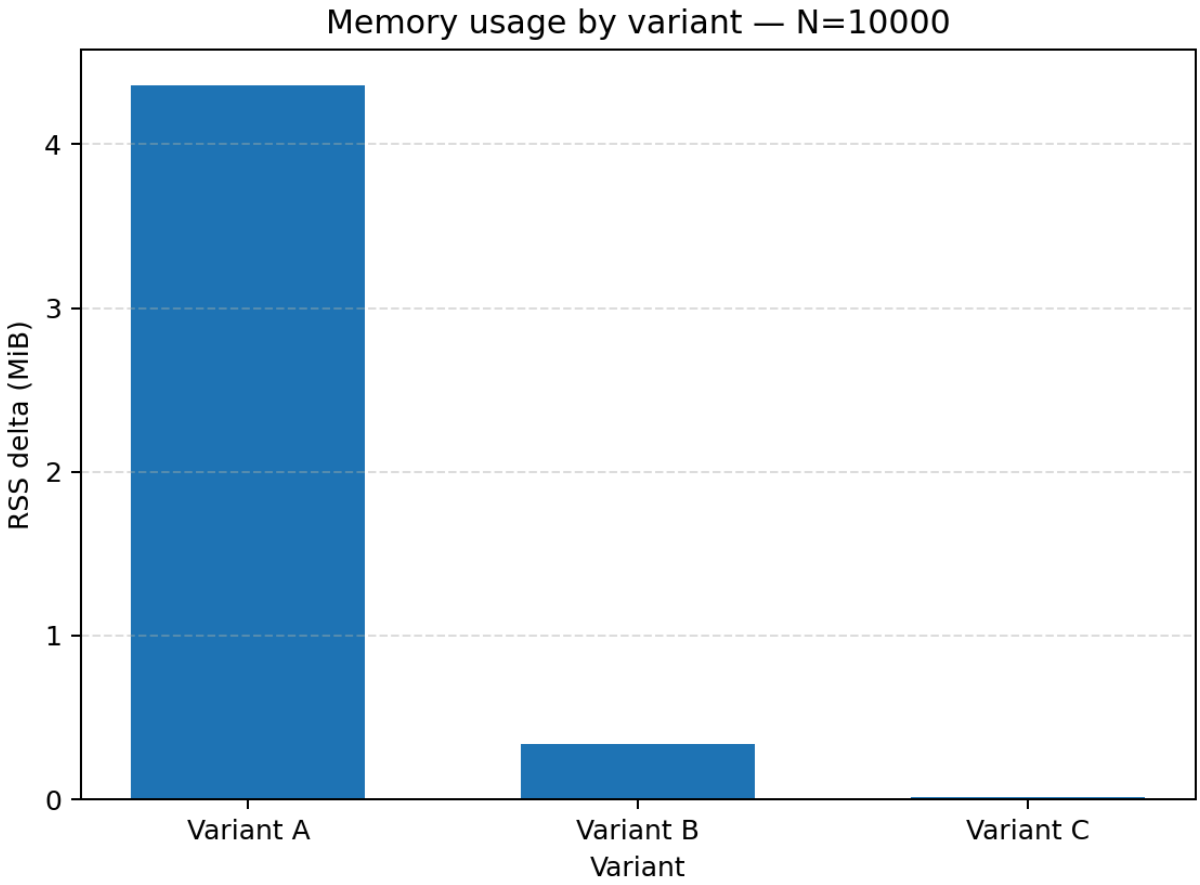
Дослідити ефективність різних реалізацій бази даних та алгоритмів сортування (std::sort, radix sort) за показниками часу, пам’яті та кількості операцій.

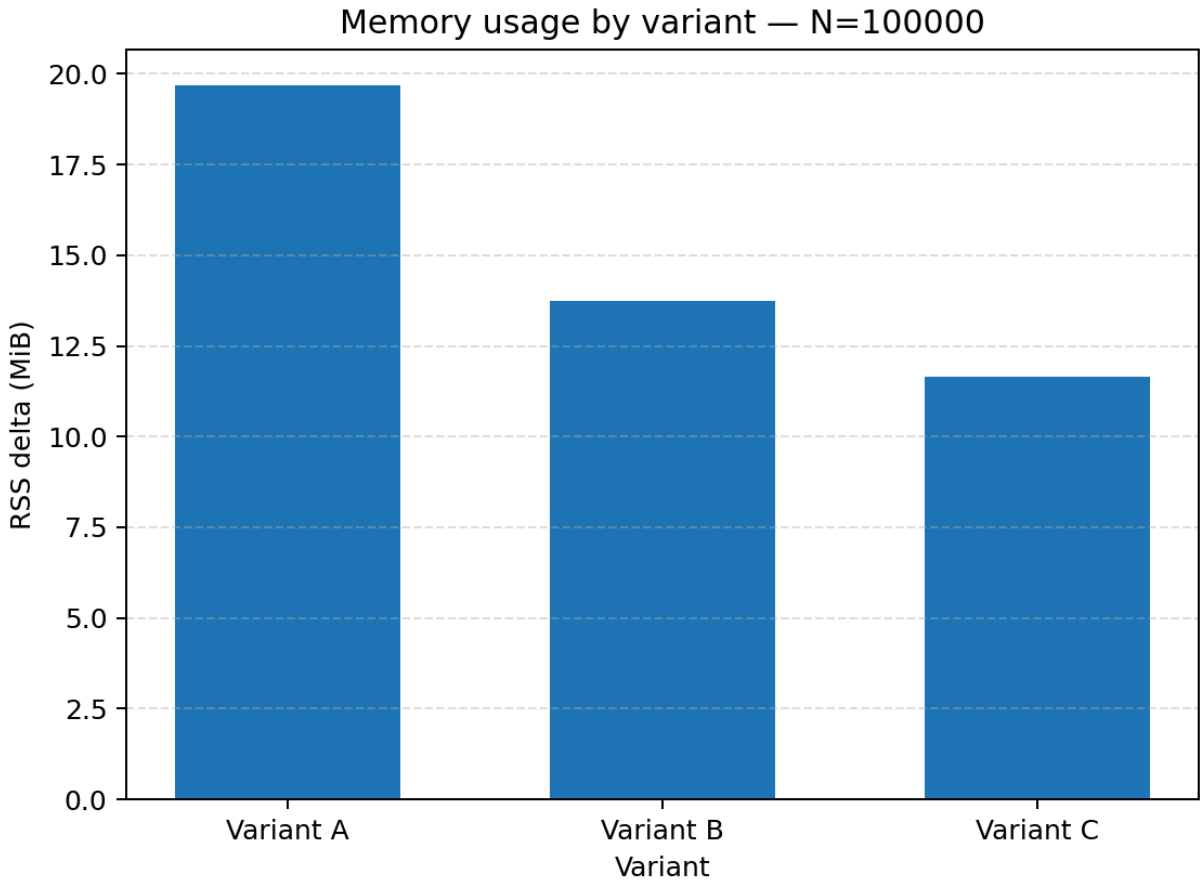
Результати експериментів

Використання пам’яті

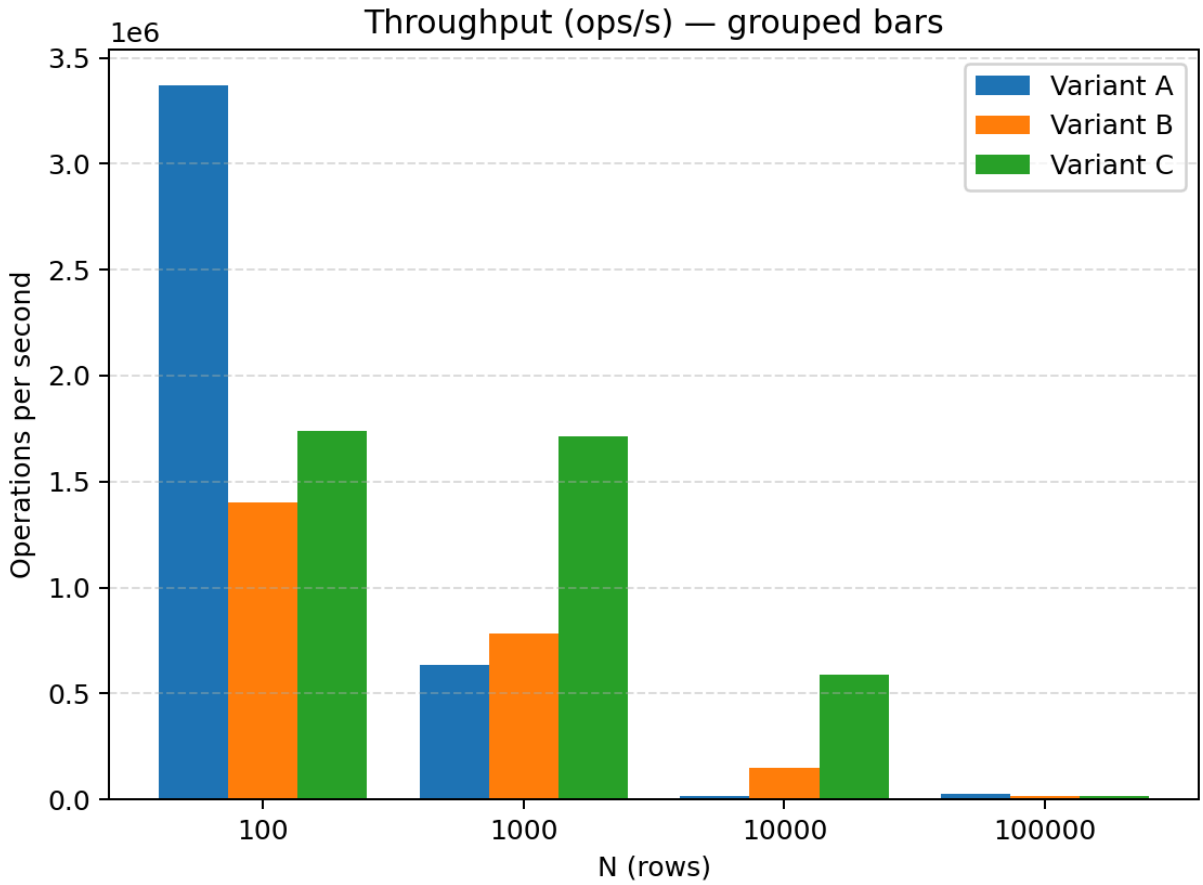

Memory usage by variant — N=100
RSS delta (MiB)
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
Variant A
Variant B
Variant C
Variant


Memory usage by variant — N=1000
RSS delta (MiB)
0.20
0.15
0.10
0.05
0.00
Variant A
Variant B
Variant C
Variant

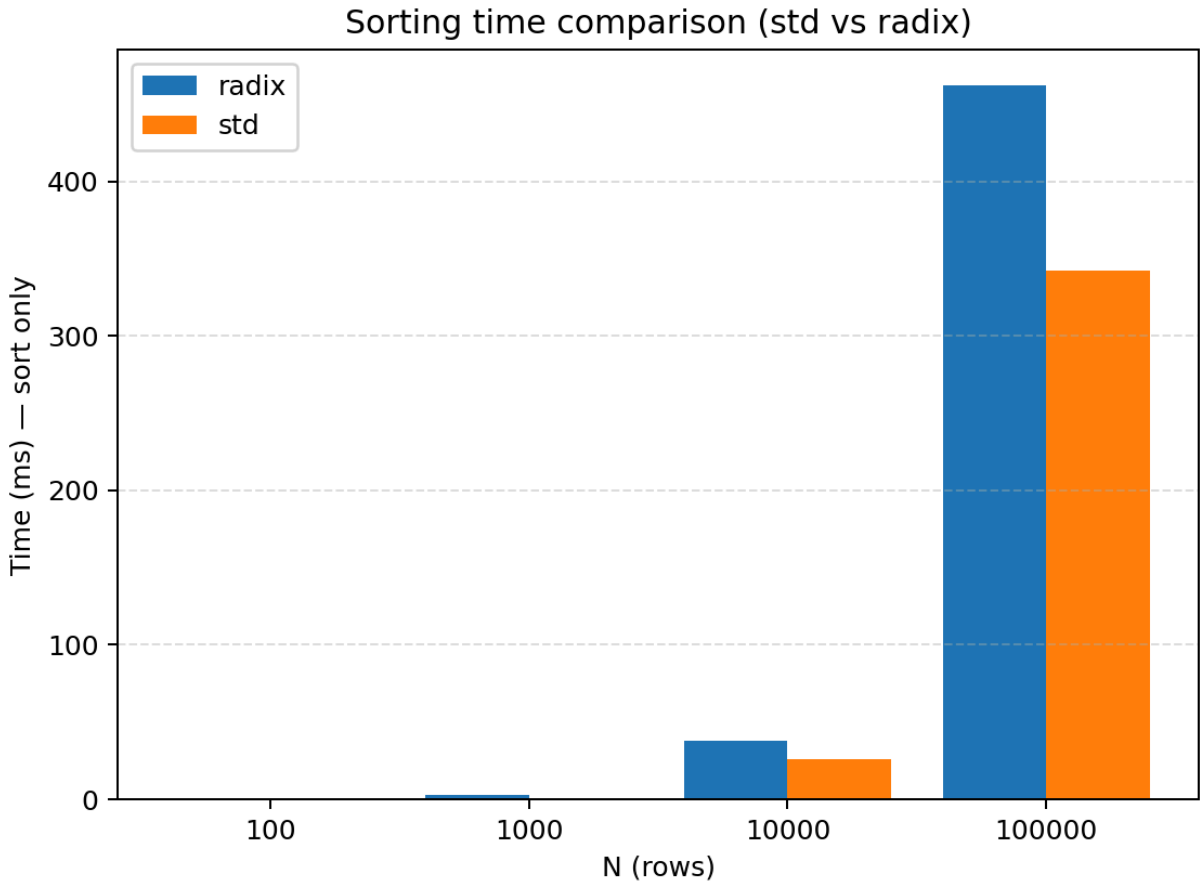

Memory usage by variant — N=10000
RSS delta (MiB)
0
1
2
3
4
Variant A
Variant B
Variant C
Variant

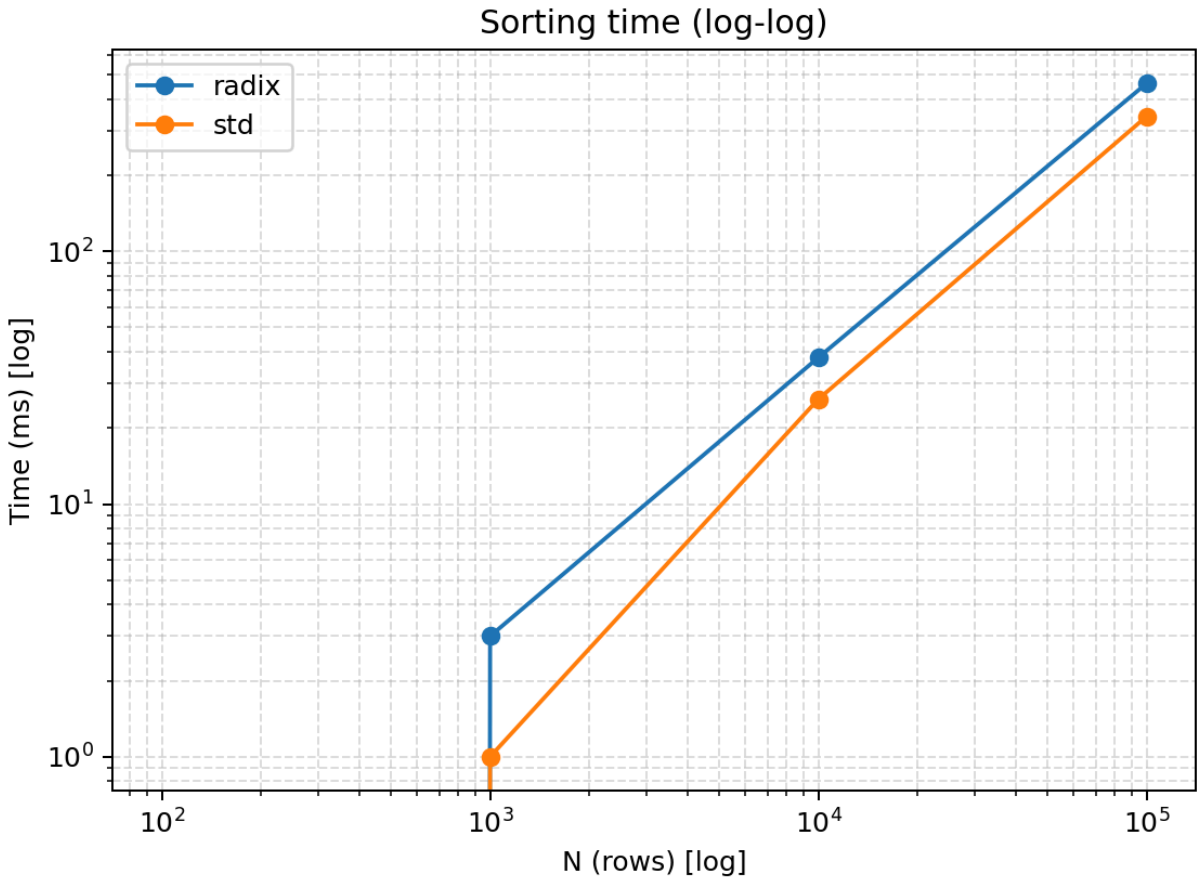


- **Графіки** `mem_N_100`, `mem_N_1000`, `mem_N_10000`, `mem_N_100000` показують, як зростає споживання пам'яті для кожного варіанта БД при збільшенні кількості записів.
- **Висновки:**
  - Variant A (на основі `unordered_map`) споживає найбільше пам'яті через хеш-таблиці та резервування місця.
  - Variant B — помірне використання (збалансоване дерево `map`).
  - Variant C — найекономніший, бо замість словників використовує щільні вектори індексів для днів народження.



- **Графік показує** продуктивність при 3 типах операцій: пошук за днем народження, зміна групи за email, пошук групи з найбільшою кількістю збігів.
- **Висновки:**
  - Найвища швидкість у Variant C завдяки векторним bucket-структурам.
  - Variant A швидший за B при великих обсягах даних (через O(1) хеш-пошук проти O(log n) дерева).
  - Variant B показує найменшу продуктивність при збільшенні розміру БД.


Sorting time comparison (std vs radix)
radix
std
Time (ms) — sort only
0
100
200
300
400
100
1000
10000
100000
N (rows)



Порівняння алгоритмів сортування

- `sort_time_bars.png` — стовпчикова діаграма часу сортування `std::sort` vs `radix sort` для різних `N`.
- `sort_time_loglog.png` — логарифмічна шкала для наочного порівняння масштабів.

Висновки:

- Для малих масивів (`N ≤ 1000`) `std::sort` (побудований на introsort) працює швидше через менший оверхед.
- Для великих (`N ≥ 10000`) — `radix sort` починає випереджати `std::sort`, особливо на даних із короткими числовими ключами (телефонами).
- У лог-лог масштабі видно, що radix-алгоритм має ближчу до лінійної складність, тоді як std::sort демонструє O(N log N).

Загальні висновки

- **Варіант C** найефективніший за швидкодією та пам’яттю завдяки векторним структурам, які мінімізують накладні витрати.
- **Варіант A** підходить для великих баз даних, якщо важливий швидкий пошук (але за рахунок більшої пам’яті).
- **Варіант B** — найповільніший, проте стабільний і детермінований за порядком елементів.
- **Radix sort** ефективніший на великих масивах, але `std::sort` універсальніший і краще працює для малих N.
- Отримані результати підтверджують очікування: вибір структури даних суттєво впливає на продуктивність як у часі, так і в пам’яті.